MiniShell

A Shell, CPU Scheduling Simulator, and Deadlock Simulator in C

CmpSc 123 Operating Systems · Group Final Project · November 2026

Queypo, Ckhiel Joshua · Viloria, Julian Shaun

Repository: github.com/CyJul17/Micro_Shell

1. Design Overview

MiniShell is a command-line shell written in C that also simulates two core parts of an operating system: CPU scheduling and deadlock handling. It runs as a loop that shows a `minishell>` prompt, reads a line, runs it, and repeats until the user types `exit`. Real commands are run with the same system calls a real shell uses (`fork`, `execvp`, `pipe`, `dup2`, `waitpid`), so the shell is not a simulation; only the scheduler and the deadlock module are simulators. The two simulators are built-in commands, `sched` and `deadlock`, that read plain text input files.

Modules

Source file	Responsibility
`main.c`	Entry point and REPL loop; interactive, `-c "command"`, and script-file modes
`parser.c`	Splits `;` `&&` `||`, tokenizes words, quotes, `$VAR` and operators, builds pipeline stages, and executes them
`builtin.c`	File and directory built-ins, `chmod`, `ls -l`, `grep`, `env`, `echo`, `exit`, `help`
`scheduler.c`	Process loader, PCB table, six scheduling algorithms, Gantt chart, state log, averages
`deadlock.c`	Matrix loader, deadlock detection, Banker's safety check, resource requests, recovery
`utils.c`	Memory, string, and path helpers; recursive copy and remove

How a command line is executed

1. **Read.** The REPL reads one line (`shell_getline`).
2. **Split.** The line is split on `;`, `&&`, and `||` outside quotes. `&&` runs the next part only if the last status was 0; `||` only if it was not.
3. **Tokenize.** Each part becomes tokens. Quotes and backslashes are removed, `$NAME`, `${NAME}`, `$?`, and `~` are expanded, and operators (`|`, `>`, `>>`, `<`, `2>`, `2>&1`, `>&2`) are marked as operators so a quoted `"|"` stays a normal word.
4. **Parse.** Tokens are cut at `|` into pipeline stages. Each stage gets its argument list and its redirections.
5. **Execute.** `cd` and `exit` run inside the shell because they change its state. Every other command, built-in or external, runs in a child created with `fork`; the child applies redirections with `dup2` and then runs the built-in or calls `execvp`. A pipeline creates one child per stage connected by pipes.
6. **Wait.** The shell waits for the children and stores the exit status in `$?`. Errors print a message and the loop continues, so the shell never crashes.

2. Modules

2.1 Module 1: Shell Core

Requirement	Implementation
File and directory built-ins	`pwd` `cd` `ls` `mkdir [-p]` `rmdir` `touch` `cp [-r]` `mv` `rm [-r -f]` `cat`
Permissions	`chmod 755 file` (numeric modes); `ls -l` prints `-rwxr-xr-x`, links, owner, size, and date
Redirection	`> >> < 2> 2>> 2>&1 >&2`, applied left to right as in Bash
Pipes	Any number of stages, e.g. `ls -l | grep txt | cat`
Variables	`NAME=value`, `$NAME`, `${NAME}`, `$?`, `env`; `NAME=value cmd` sets it for that command only
Conditionals	`cmd1 && cmd2`, `cmd1 || cmd2`, `cmd1 ; cmd2`
Text search	`grep [-i] [-n] pattern [file...]` for plain text; `grep -E` adds regular expressions (`. [a-z] * + ?` escaping, groups) using the POSIX regex library
Error handling	Clear messages for unknown commands (status 127), missing files, bad options and syntax errors; Ctrl+C stops the running command, not the shell

2.2 Module 2: Process Scheduling Simulator

`sched <algorithm> [--quantum N] <file>` loads processes (one per line: `PID arrival burst priority`; `-` reads from standard input). Each process has a PCB with its PID, arrival, burst, priority, remaining time, completion, waiting, turnaround, queue level, and state. The simulator advances a clock, records every CPU slice in a Gantt chart, and computes **Turnaround = Completion − Arrival** and **Waiting = Turnaround − Burst** with their averages.

Algorithm	Selection rule	Preemption
FCFS	Earliest arrival (ties by PID)	Non-preemptive
SJN	Shortest burst among ready processes	Non-preemptive
Priority	Smallest priority number	Non-preemptive
Round-Robin	FIFO ready queue	Quantum 4 (`--quantum N`)
MLQ	Q0: priority 0-2, Q1: 3-5, Q2: 6+	Q0 q=2, Q1 q=4, Q2 FCFS
MLFQ	All start in Q0; demoted after a full quantum	q = 2, 4, 8

Each run prints the initial PCB table (all **New**), the Gantt chart, a timed log of PCB state changes, the final PCB table (all **Terminated**), and the per-process results. The state log shows **New → Ready** on arrival, **Ready → Running** on dispatch, **Running → Ready** on preemption, and **Running → Terminated** on completion. There are no I/O events, so no process enters **Waiting**. Round-Robin on `basic.txt` begins:

```
t=0     P1    New      -> Ready      (arrived)
t=0     P1    Ready    -> Running    (dispatched)
t=1     P2    New      -> Ready      (arrived)
t=4     P1    Running  -> Ready      (preempted)
t=4     P2    Ready    -> Running    (dispatched)
t=8     P2    Running  -> Terminated (burst complete)
```

2.3 Module 3: Deadlock Simulator

`deadlock detect|banker <file>` reads the number of processes and resources, the **Allocation** and **Max** matrices, and the **Available** vector, and computes **Need = Max − Allocation**. Both algorithms use the same reduction: start with **Work = Available**, repeatedly find an unfinished process whose demand fits in Work, let it finish, and add its allocation back to Work.

- **Detection** uses each process's current request (an optional `requests` matrix; without one it assumes each process requests its full Need). Processes that can never finish are reported as deadlocked, followed by a recovery suggestion (terminate a process or preempt its resources).
- **Banker's algorithm** uses Need. If every process can finish, the state is **SAFE** and the order found is printed as the safe sequence; otherwise it is **UNSAFE**.
- **Resource request** (`--request PID r1 r2 ...` or a `request` line in the file) is granted only if it is ≤ the process's Need, ≤ Available, and the state after pretending to grant it is still safe. Otherwise it is denied with the reason, and the state is rolled back.

3. Module 4: Comparison

3.1 Scheduling algorithms

The same two input files were run through all six algorithms (Round-Robin with quantum 4). Every value below comes from the simulator's own output, collected by `tests/compare.sh`.

Algorithm	mixed.txt avg waiting	mixed.txt avg turnaround	basic.txt avg waiting	basic.txt avg turnaround
FCFS	5.20	9.20	8.75	15.25
SJN	**3.80**	**7.80**	**7.75**	**14.25**
Priority	5.40	9.40	**7.75**	**14.25**
Round-Robin (q=4)	5.60	9.60	11.75	18.25
MLQ	4.80	8.80	9.50	16.00
MLFQ	7.40	11.40	13.00	19.50

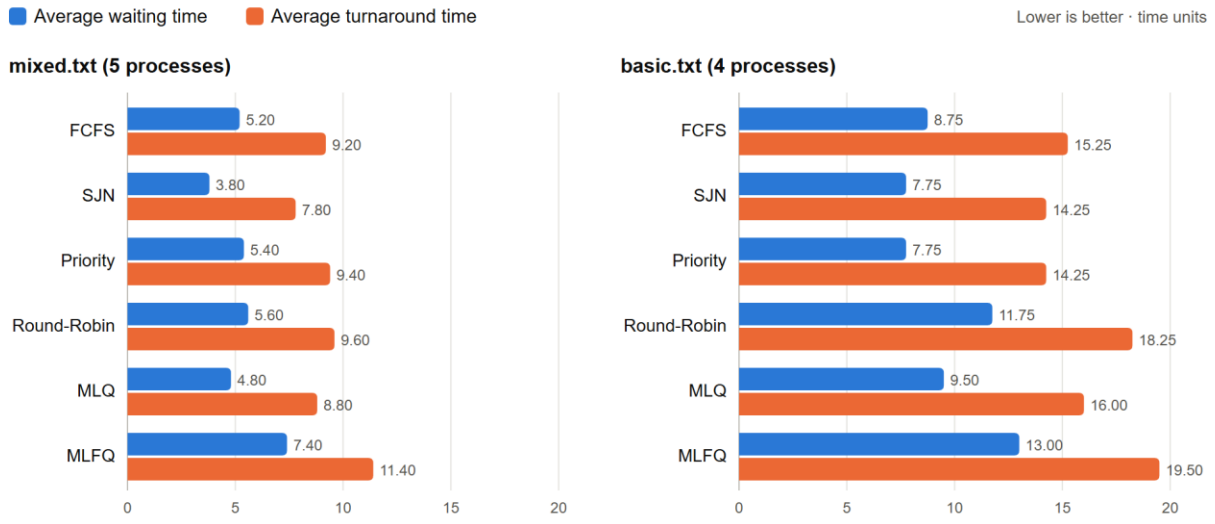


Figure 1. Average waiting and turnaround time per algorithm (lower is better).

Gantt charts for `mixed.txt`:

```
FCFS      [0-3] P1 | [3-9] P2 | [9-13] P3 | [13-15] P4 | [15-20] P5
SJN       [0-3] P1 | [3-7] P3 | [7-9] P4 | [9-14] P5 | [14-20] P2
Priority  [0-6] P2 | [6-8] P4 | [8-11] P1 | [11-16] P5 | [16-20] P3
RR (q=4)  [0-3] P1 | [3-7] P2 | [7-11] P3 | [11-13] P4 | [13-15] P2 | [15-20] P5
```

Best algorithm: SJN. It had the lowest average waiting and turnaround time on both workloads (3.80 / 7.80 on `mixed.txt`). Running the shortest ready job first means the most processes finish early, so less total time is spent waiting. In `mixed.txt` it moves the short jobs P3 and P4 ahead of the long job P2, while FCFS makes them wait behind P2 (the convoy effect). The trade-off is that SJN needs to know burst times in advance and can starve long jobs: P2 finishes last.

- **Priority** tied SJN on `basic.txt` because there the priorities happened to pick the same short jobs first; on `mixed.txt` the high-priority P2 is also the longest job, so it did worse.
- **MLQ** came second on `mixed.txt` by giving high-priority short jobs a fast queue.
- **Round-Robin** gives the fairest response time, but splitting jobs into slices delays their completion, which raises the averages.
- **MLFQ** had the highest averages: every job starts in Q0 and is demoted after each full quantum, so long jobs are split many times and wait behind newer ones.

3.2 Deadlock: safe and unsafe examples

Safe (`safe.txt`, 5 processes, 3 resources, Available = 3 3 2): P1's Need (1 2 2) fits in Available, so P1 can finish and release its resources, which then lets P3, P0, P2, and P4 finish. Output: **State is SAFE. Safe sequence: P1 → P3 → P0 → P2 → P4.** A request of (1, 0, 2) by P1 is **GRANTED**; a request of (7, 4, 3) by P0 is **DENIED** because only 3 units of resource A are free.

Unsafe (`unsafe.txt`, 3 processes, 2 resources, Available = 0 0):

Process	Allocation	Max	Need
P0	1 0	2 1	1 1
P1	0 1	1 2	1 1
P2	1 1	2 2	1 1

Every process still needs one more unit of each resource and none are free, so no process can finish. Output: **State is UNSAFE. No safe sequence exists. Unfinished processes: P0 P1 P2.** Unsafe does not yet mean deadlocked; it means the system can no longer guarantee that all processes finish, which is exactly what the Banker's algorithm prevents by denying requests that lead there. With a `requests` matrix, detection on `requests_deadlock.txt` finds P1, P2, P3, and P4 deadlocked while P0 can finish.

4. Testing

`make test` runs 126 automated checks (`tests/run_tests.sh`) covering every built-in, redirection, pipes, variables, conditionals, `grep`, error cases, all six schedulers, and every deadlock input. The simulators have 5 process files and 8 resource files, including edge cases: an empty process list, a single process, an idle CPU before the first arrival, and a single-process deadlock system. Each input, its purpose, and its verified result are listed in `docs/TEST_CASES.md`.

5. Member Contributions

Member	Work
Queypo, Ckhiel Joshua	Shell core: parser, executor, built-ins, redirection, pipes, variables, error handling, integration of all commands; bug fixes and PCB state transitions; test runner and test-case catalog
Viloria, Julian Shaun	Scheduler (six algorithms, PCB table, Gantt chart), deadlock detection and Banker's algorithm, test input files, comparison data, README and documentation

Both members tested and reviewed each other's modules and the combined program.